Book-Exchange Platform: Sprint 1 Task Plan

Sprint 1: 27–31 Oct 2026. 31 Oct is a Saturday, so aim to finish by Fri 30 Oct and keep Saturday as buffer.

Roles: Abhiram = repo owner / DevOps, Nishkal = Auth, Manoj = Listings, Ravish = UI / docs / demo.

Everyone, today (Tue 27 Oct)

1. Create a GitHub account if needed and send your username to Abhiram.
2. Accept the repo and Jira invites.
3. Clone the repo, then read `docs/SRS.md` (§4 to §5) and `docs/SAD.md` (§3 and §4.3).
4. Always branch as `BEP-<jira-number>-short-name`, open a PR that mentions the Jira key, and never push to `main`.
5. Move your Jira cards daily: To Do → In Progress → In Review → Done.

Abhiram (repo owner / DevOps)

Today

1. Create the GitHub repo and add `/docs` (SRS, SAD, Test Plan). Add all 3 teammates as collaborators.
2. Paste the repo link in the TA's spreadsheet.
3. Create the Jira project and add the team. Create Sprint 1 (27–31 Oct) and Sprint 2 (2–7 Nov).
4. Create the epics (Auth, Listings, Requests & Matching, Exchange, Admin, DevOps) and one story per FR/SR from the SRS. Assign story points and owners.
5. Protect `main`: require 1 PR review and passing CI.

Wed 28

1. Scaffold `backend/`, `frontend/`, `tests/`, `docker-compose.yml` (FastAPI, React, Postgres) and push to `main`.
2. Write the Alembic migration for `users`, `books` and `listings`, and enable `pg_trgm`.
3. Write the GitHub Actions workflow: lint → pytest with coverage (fail under 70%) → Docker build → deploy stub.

Thu 29 – Fri 30

1. Review and merge PRs promptly. Do not review your own.

2. Deploy to Render or similar with HTTPS, and add the deploy stage to CI.
3. Add a `/health` endpoint.
4. Write a README with setup and the `docker compose up` command.

Sat 31

1. Close Sprint 1 in Jira.
2. Screenshot the burndown and save it to `/docs`.

Nishkal (Auth)

1. **Wed:** Build the `POST /auth/register` endpoint (F-001). It should accept only a university email domain, return 409 on a duplicate email, and hash passwords with bcrypt (SR-001).
2. **Wed:** Write pytest for TC-AUTH-01/02 and TC-SEC-01.
3. **Thu:** Build `POST /auth/login` (F-002). It should issue a JWT valid for 30 minutes (SR-002).
4. **Thu:** Add the login rate limit, where the 6th failed attempt in 15 minutes returns 429 (SR-006).
5. **Thu:** Write pytest for TC-AUTH-03/04 and TC-SEC-02/06.
6. **Fri:** Build logout and expiry handling (F-003).
7. **Fri:** Build `GET/PUT /users/me` (F-004).
8. **Fri:** Write pytest for TC-AUTH-05/06 and TC-PRF-01.
9. Raise one PR per feature.

Manoj (Listings)

1. **Wed:** Build the SQLAlchemy models and repository layer for books and listings. Use parameterised queries only (SR-004).
2. **Wed:** Build `POST /listings` (F-010) with server-side validation. Upsert the book by ISBN.
3. **Wed:** Write pytest for TC-LST-01/02.
4. **Thu:** Build `GET /listings` (F-011) with filters on `q`, `isbn` and `location`, plus pagination. Add indexes on `isbn` and a trigram index on `title`.
5. **Thu:** Write pytest for TC-LST-03/04.
6. **Fri:** Build `PUT /listings/{id}` (F-012) and `DELETE /listings/{id}` (F-013). Updates are owner-only (403 otherwise). Delete is a soft delete and returns 409 when an exchange is active.

7. **Fri:** Write pytest for TC-LST-05/06/07 and TC-SEC-04/05.
8. **Fri:** Write the Locust script and run it on 10,000 seeded listings (TC-PERF-01).
9. Add the seed script to `/tests`.

Ravish (UI / docs / demo)

1. **Wed:** Set up the React (Vite) app with routing, an Axios client and a layout with a navbar. Make it responsive from 360 px.
2. **Wed – Thu:** Build the Register and Login pages with inline validation. Store the JWT and handle 401 and 429 errors.
3. **Thu:** Build the Browse/Search page with pagination and an empty-state message.
4. **Fri:** Build the My Listings page, covering create, edit and delete.
5. **Fri:** Build the Profile page.
6. **Fri:** Run TC-UX-01 manually and tick it off.
7. **Fri:** Update the RTM status column in `docs/SRS.md` from N to A for the tests that pass.
8. **Fri:** Note any deviations from the SRS/SAD in `docs/DEVIATIONS.md`.
9. **Fri/Sat:** Record the 1-minute demo (register → login → create listing → search). Save it as `/demo/sprint1.mp4` and push it.

Whole team, Fri 30 – Sat 31

1. Merge all PRs and confirm CI is green on `main`.
2. Run `docker compose up` on a clean machine (TC-PORT-01).
3. Check the deployed link works.
4. Confirm the backlog for Sprint 2 is assigned in Jira.

Sprint 2 (2–7 Nov) covers requests, the matching engine, exchanges, ratings and admin.

Detailed Plan

This part expands the plan above into concrete tasks, files, commands and checks. Sprint 1 runs Tue 27 Oct to Sat 31 Oct. Target: everything merged by Fri 30 Oct, 18:00.

Schema additions to agree on now

Three small additions to the SAD data model are needed in Sprint 1. Abhiram puts them in the first migration; Ravish logs them in `docs/DEVIATIONS.md`.

- `users.token_version` (int, default 0): incremented on logout so earlier JWTs are rejected (F-003).
- `users.department` (string, nullable): profile field (F-004).
- `listings.deleted_at` (timestamp, nullable): soft delete (F-013).

One endpoint is also added: `GET /api/v1/listings/mine` (the user's own listings, needed by the My Listings page).

Handoffs (who waits on whom)

When	From	To	What
Tue evening	Abhiram	Everyone	Repo, collaborator access, Jira board and branch protection live
Wed 12:00	Abhiram	Everyone	Scaffold, docker compose, models and base migration merged to `main`
Wed 15:00	Nishkal	Manoj, Ravish	`core/security.py`, `core/deps.py` (`get_current_user`) and `core/errors.py` merged
Thu 12:00	Nishkal	Ravish	Register and login endpoints working on `main`
Thu 18:00	Manoj	Ravish	`POST /listings`, `GET /listings` and `GET /listings/mine` working
Fri 12:00	Manoj	Ravish	`PUT` and `DELETE /listings/{id}` working
Fri 14:00	Everyone	Abhiram	All PRs raised
Fri 18:00	Abhiram	Everyone	All PRs reviewed and merged; regression run starts

Jira backlog to create today

Every story gets the Req ID in its title, the acceptance criteria copied from the SRS, and its test case IDs from the Test Plan. Tests are part of each story, not separate cards.

Sprint 1 (27 to 31 Oct)

#	Story	Req	Owner	Points
1	Repo scaffold, docker compose, models, base migration	NF-006	Abhiram	5
2	CI pipeline: lint, test with coverage, build, deploy stage	NF-004	Abhiram	5
3	Cloud deployment with HTTPS and health endpoint	NF-002, SR-003	Abhiram	3
4	Register	F-001, SR-001	Nishkal	3
5	Login and JWT	F-002, SR-002	Nishkal	3
6	Login rate limit	SR-006	Nishkal	2
7	Logout and token expiry	F-003	Nishkal	2
8	Profile view and edit	F-004	Nishkal	2
9	Create listing	F-010, SR-004	Manoj	3
10	Search listings with indexes	F-011, NF-001	Manoj	5
11	Update listing, owner only	F-012, SR-005	Manoj	2
12	Delete listing (soft delete)	F-013	Manoj	2
13	Seed script and Locust performance test	NF-001	Manoj	3
14	UI shell, routing, Register and Login pages	F-001, F-002, NF-003	Ravish	5
15	UI Browse and Search page	F-011	Ravish	3
16	UI My Listings and Profile pages	F-004, F-010, F-012, F-013	Ravish	5
17	Demo video, RTM update, DEVIATIONS.md	NF-003	Ravish	2

Totals: Abhiram 13, Nishkal 12, Manoj 15, Ravish 15.

Sprint 2 (2 to 7 Nov), create now, assign on 2 Nov

Story	Req	Points
Create borrow request	F-020	3
Cancel request	F-021	2
Matching engine (scoring, lock, re-match)	F-022	8
In-app notifications	F-023	3
Accept and decline match	F-024	5
Exchange lifecycle and due date	F-025	5
Confirm return	F-026	2
Exchange history	F-030	2
Ratings	F-031	3
Admin moderation and audit log	F-040	5
Contact privacy until mutual accept	SR-007	3
TLS verification and uptime report	SR-003, NF-002	2
Matching performance benchmark	NF-005	3
Final documentation, deviations, demo video	All	3

Day-by-day overview

Day	Abhiram	Nishkal	Manoj	Ravish
Tue 27	Repo, Jira, branch protection, PR template	Local setup, read auth docs	Local setup, read listing docs	Local setup, create Vite app
Wed 28	Scaffold and models (AM), CI (PM)	Security utils and deps (AM), Register (PM)	Create listing	App shell, Register and Login pages
Thu 29	Deploy, reviews	Login and rate limit	Search, indexes, `/mine`	Browse page wired to API

Day	Abhiram	Nishkal	Manoj	Ravish
Fri 30	Reviews, merge, regression, README, tag	Logout and profile, final tests	Update, delete, seed, perf test	My Listings, Profile, RTM, demo video
Sat 31	Close sprint, burndown screenshot	Buffer	Buffer	Buffer

Abhiram: repo owner and DevOps (13 points)

Tue 27 Oct: repo, Jira, rules

1. Create the repo `book-exchange-platform` on GitHub with a README and a `.gitignore` for Python and Node.
2. Settings, Collaborators: add Nishkal, Manoj and Ravish.
3. Fill in the team names in `docs/SRS.md`, `docs/SAD.md` and `docs/Test_Plan.md`, then commit all three to `main` under `docs/`. This is the last direct push to `main`.
4. Add `.github/pull_request_template.md` with: Jira key, what changed, how to test, and a checklist (tests added, CI green, docs updated).
5. Settings, Branches, add a rule for `main`: require a pull request, require 1 approval, block force pushes, and apply the rules to admins too. Add the required status checks after CI exists (Wed).
6. Paste the repo link in the TA's spreadsheet. One link only.
7. Create the Jira project (Scrum template). Board columns: To Do, In Progress, In Review, Done. Turn on story points as the estimate.
8. Create the six epics: Auth, Listings, Requests and Matching, Exchange, Admin, DevOps.
9. Create every story from the two backlog tables above. Put the Req ID in the title, copy the acceptance criteria from the SRS, and assign owners and points.
10. Create Sprint 1 (27 to 31 Oct) and Sprint 2 (2 to 7 Nov). Move the Sprint 1 stories into Sprint 1 and start it. The burndown chart only works if points are set before the sprint starts.
11. Optional: install the GitHub for Jira app so branches and PRs containing the Jira key show up on the cards.
12. Message the team: branch names `BEP-<jira-number>-short-name`, commit messages `BEP-12: add login endpoint`, no pushes to `main`.

Wed 28, before 12:00: scaffold (story 1)

Target layout:

```
book-exchange-platform/
  backend/
    app/ core/ db/ models/ schemas/ repositories/ services/ routers/
    scripts/
    tests/
    requirements.txt  Dockerfile  alembic.ini
  frontend/
  tests/perf/
  docs/
  .github/workflows/ci.yml
  docker-compose.yml
  .env.example
```

1. `requirements.txt`: fastapi, uvicorn, sqlalchemy, psycopg2-binary, alembic, pydantic[email], pyjwt, passlib[bcrypt], pytest, pytest-cov, httpx, ruff.
2. `app/main.py`: FastAPI app, routers mounted under `/api/v1`, and `GET /health` returning `{status: ok}`. Add CORS middleware with the frontend origin from an env var.
3. `app/core/config.py`: read `DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRE_MINUTES` (default 30) and `ALLOWED_EMAIL_DOMAINS` from the environment. Commit `.env.example` with dummy values, never the real `.env`.
4. `app/db/session.py`: engine, `SessionLocal`, and the `get_db` dependency.
5. Models, all in one PR so nobody conflicts: `User` (id, name, email unique, password_hash, role, department, location, avg_rating, active, token_version), `Book` (id, isbn, title, author), `Listing` (id, owner_id, book_id, condition_rank, location, status, created_at, deleted_at).
6. Alembic: `alembic init`, point `env.py` at the models' `Base`, run `alembic revision --autogenerate -m initial`, and add `CREATE EXTENSION IF NOT EXISTS pg_trgm` at the top of `upgrade()`.
7. `docker-compose.yml`: `db` (postgres:15 with a volume and healthcheck), `backend` (build `./backend`, waits for `db`, runs `alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000`), and `frontend` (placeholder until Ravish's Dockerfile arrives).
8. Check `docker compose up --build`, then open `localhost:8000/docs` and `localhost:8000/health`.
9. Push branch `BEP-<n>-scaffold`, open the PR and ask Nishkal to review (you cannot review your own). Merge by 12:00 and tell the team.

Wed 28, afternoon: CI pipeline (story 2)

Create `.github/workflows/ci.yml`, triggered on push and pull request to `main`, with four jobs:

1. `backend`: Postgres 15 service container; checkout; Python 3.11 with pip cache; install; `ruff check backend`; `alembic upgrade head`; `pytest --cov=app --cov-fail-under=70`; upload the coverage report as an artifact; run `pip-audit` on `requirements.txt` (allowed to warn).
2. `frontend`: Node 20; `npm ci`; `npm run build`.
3. `docker`: needs the two jobs above; `docker compose build`.
4. `deploy`: needs `docker`; runs only when `github.ref == 'refs/heads/main'`. For now it just echoes; Thursday replaces it with the real deploy.

When the first run is green, add `backend`, `frontend` and `docker` as required status checks in the branch protection rule. Add the CI status badge to the README.

Thu 29: deployment and monitoring (story 3)

1. On Render (or an equivalent free host), create a PostgreSQL instance and copy its connection string.
2. Create a web service from the repo's `backend` Docker image. Set env vars `DATABASE_URL`, `JWT_SECRET` (random, 32+ characters), `ALLOWED_EMAIL_DOMAINS` and the CORS origin.
3. Create a static site for the frontend: build command `npm ci && npm run build`, publish directory `dist`, env var `VITE_API_URL`.
4. Save the host's deploy hook URL as the GitHub secret `RENDER_DEPLOY_HOOK`. Replace the echo in the `deploy` job with `curl -X POST ${{ secrets.RENDER_DEPLOY_HOOK }}`.
5. Confirm that `http://` redirects to `https://` (this is the TLS check, TC-SEC-03). Put both URLs in the README.
6. Add an uptime monitor (for example UptimeRobot) that pings `/health` every 5 minutes. Its report is the evidence for NF-002 in Sprint 2.
7. Free tiers can sleep after inactivity. Open the site a few minutes before any demo.

Review duty, Wed to Fri

Review PRs within about 3 hours. For each PR check:

- Jira key is in the title and CI is green.

- Every acceptance criterion has a test.
- No secrets, passwords or tokens in code or logs.
- Database access is parameterised (ORM or bound parameters only).
- Mutating endpoints check ownership or role.
- Response models never include `password_hash` or other private fields.

Use squash and merge, and delete the branch afterwards.

Fri 30 and Sat 31: close out

1. 14:00: confirm everyone's PRs are raised. 18:00: all merged.
2. Fresh-clone test on a clean folder: `cp .env.example .env`, then `docker compose up --build`. Check register, login and create listing work (TC-PORT-01).
3. Run the full test suite on `main` and confirm CI is green.
4. Tag the release: `git tag v0.1-sprint1 && git push --tags`.
5. Sat: complete Sprint 1 in Jira. Move unfinished cards to Sprint 2 with a comment. Open Reports, Burndown Chart, screenshot it and save it as `docs/sprint1-burndown.png`.

Nishkal: authentication (12 points)

Everything under `backend/app/`. Endpoint contracts are in SAD section 4.3 and the tests are in the Test Plan (TC-AUTH-*, TC-PRF-01, TC-SEC-01/02/06).

Tue 27 Oct: setup

1. Install Python 3.11 and Docker. Clone the repo and create a virtualenv.
2. Read SRS 4.1 and 5.1.2, SAD 3.9 and 4.3 (Auth), and the Auth and Security rows of the Test Plan.

Wed 28, PR 1: security utilities, dependencies, errors (merge by 15:00)

Manoj and Ravish depend on this, so do it first.

1. `core/security.py`:
   - `hash_password(plain)` and `verify_password(plain, hashed)` using passlib bcrypt.
   - `create_access_token(user_id, role, token_version)` with claims `sub`, `role`, `ver`, `iat` and `exp` (now plus 30 minutes), HS256, key from `JWT_SECRET`.

- `decode_token(token)` that raises a 401 on a bad signature or an expired token.

2. `core/deps.py`:
   - `get_current_user`: read the bearer token, decode it, load the user, and return 401 if the user is missing or inactive or `ver` does not equal the user's `token_version`.
   - `require_role('admin')` dependency factory that returns 403.
3. `core/errors.py`: an `AppError(status, code, message)` exception and a handler that returns `{error: {code, message}}`. Everyone uses this format.
4. Tests in `tests/test_security.py`: hash differs from plain text; verify works for right and wrong passwords; expired token returns 401; tampered token returns 401.

Wed 28, PR 2: register (F-001, SR-001)

1. `schemas/auth.py`: `RegisterIn` (name 1 to 100 characters, email, password of at least 8 characters containing a letter and a digit) and `UserOut` (id, name, email, role, department, location). `UserOut` must not contain the hash.
2. `repositories/user_repo.py`: `get_by_email` and `create`.
3. `services/auth_service.register`: lowercase the email, reject domains outside `ALLOWED_EMAIL_DOMAINS` with 400 `INVALID_EMAIL_DOMAIN`, reject duplicates with 409 `EMAIL_EXISTS`, hash the password, save with role `student`. Confirm your university's email domain and put it in `.env.example`.
4. `routers/auth.py`: `POST /api/v1/auth/register` returns 201 with `UserOut`.
5. Tests: TC-AUTH-01 (201 and a row exists), TC-AUTH-02 (duplicate gives 409, a non-university address gives 400), weak password gives 422, TC-SEC-01 (the stored value starts with `$2` and is not the plain password).

Thu 29, PR 3: login and rate limit (F-002, SR-002, SR-006)

1. `POST /api/v1/auth/login` with `{email, password}` returns 200 `{token, expiresIn: 1800}`. Return the same `INVALID_CREDENTIALS` 401 for an unknown email and a wrong password.
2. `core/rate_limit.py`: an in-memory dict keyed by `(email, ip)` holding failure timestamps. Before authenticating, drop entries older than 15 minutes. If 5 or more failures remain, return 429 with a `Retry-After` header. Clear the entry on a successful login. Read the limit and window from env vars so tests can shrink them.
3. Note for `DEVIATIONS.md`: the in-memory limiter resets on restart and works for a single instance only.

4. Tests: TC-AUTH-03, TC-AUTH-04, TC-SEC-02 (a protected route with a modified token gives 401), TC-SEC-06 (6th failed attempt gives 429, and even the correct password is blocked inside the window).

Fri 30, PR 4: logout and profile (F-003, F-004)

1. `POST /api/v1/auth/logout` (authenticated) increments `token_version` and returns 204. The old token then returns 401.
2. `GET /api/v1/users/me` returns `UserOut`.
3. `PUT /api/v1/users/me` accepts `{name?, department?, location?}` with validation. Email and role cannot be changed through this endpoint.
4. Tests: TC-AUTH-05 (use the token after logout, expect 401), TC-AUTH-06 (a token with a past `exp`, expect 401), TC-PRF-01 (edit, then fetch, new values returned), and a test that sending `role: admin` does not change the role.
5. Check coverage for the auth module is above 85%, and raise PRs by 14:00.

Manoj: listings (15 points)

Everything under `backend/app/` plus the seed and performance scripts. Contracts are in SAD section 4.3; tests are TC-LST-*, TC-PERF-01, TC-SEC-04/05.

Tue 27 Oct: setup

1. Install Python 3.11 and Docker. Clone the repo.
2. Read SRS 4.2 and NF-001, SAD 3.5, 3.6 and 4.3 (Listing Service), and the Listing and Performance rows of the Test Plan.

Wed 28, PR 1: create listing (F-010, SR-004)

Start after Abhiram's scaffold lands at 12:00. Until Nishkal's `get_current_user` is merged (about 15:00), develop against a temporary stub and swap it in afterwards.

1. `schemas/listing.py`: `ListingCreate` with title (1 to 200 characters), author (1 to 150), ISBN (required, 10 or 13 digits after removing hyphens, stored as digits only), condition (`NEW`, `GOOD`, `FAIR`, `POOR`) and location (1 to 100). Also `ListingOut` with id, title, author, isbn, condition, location, status, ownerId and createdAt.
2. `repositories/book_repo.py`: `get_or_create_by_isbn(isbn, title, author)`. `repositories/listing_repo.py`: `create`.

3. `services/listing_service.create`: map condition to rank (`NEW` 4, `GOOD` 3, `FAIR` 2, `POOR` 1), set the owner to the current user and the status to `AVAILABLE`.
4. `routers/listings.py`: `POST /api/v1/listings` returns 201 with `ListingOut`. Use ORM or bound parameters only.
5. Tests: TC-LST-01 (201, status AVAILABLE), TC-LST-02 (empty title, bad ISBN and bad condition each give a validation error naming the field), and an unauthenticated call gives 401.

Thu 29, PR 2: search, own listings, indexes (F-011, NF-001)

1. `GET /api/v1/listings?q=&isbn=&location=&page=1&size=20`. No login needed. Return only listings with status `AVAILABLE` and `deleted_at IS NULL`.
2. For `q`, match title or author with `ILIKE`, and order by trigram `similarity(title, q)` descending, then newest first. `size` is capped at 100 (422 above that). Response: `{items, page, size, total}`.
3. `GET /api/v1/listings/mine` (authenticated): the current user's own non-deleted listings, including unavailable ones. Register this route before `/listings/{id}` so the path is not swallowed.
4. New Alembic migration `add_search_indexes`: index on `books(isbn)`, GIN trigram index on `books(title)` (`gin_trgm_ops`), index on `listings(status)`.
5. Tests: TC-LST-03 (query returns only matches), TC-LST-04 (unknown query returns an empty list; page 2 returns the correct slice), plus a test for the `size` cap.
6. Tell Ravish when this is on `main`.

Fri 30, PR 3: update and delete (F-012, F-013, SR-005)

1. `PUT /api/v1/listings/{id}`: owner only, otherwise 403 `NOT_OWNER`. Return 404 for a missing or deleted listing. Allow partial updates, including switching status between `AVAILABLE` and `UNAVAILABLE`.
2. `DELETE /api/v1/listings/{id}`: owner only. If `has_active_exchange(listing_id)` is true, return 409 `ACTIVE_EXCHANGE`. Otherwise set `deleted_at` and return 204.
3. In Sprint 1 there is no exchange table, so `has_active_exchange` returns False. Sprint 2 makes it a real query. TC-LST-07 patches it to return True.
4. Tests: TC-LST-05 (owner 200, other user 403), TC-LST-06, TC-LST-07, TC-SEC-04 (payloads such as `' OR 1=1 --` in `q`, title and location cause no error and return no extra rows), TC-SEC-05 (non-owner update and delete give 403).

Thu PM to Fri: seed data and performance test (TC-PERF-01)

1. `backend/scripts/seed.py`: create 20 users with a known test password, then about 2,000 books and 10,000 listings, inserted in batches of 1,000 per commit. Run it with `docker compose exec backend python scripts/seed.py`.
2. `tests/perf/locustfile.py`: one task that calls `GET /api/v1/listings?q=<random word from the seeded titles>`.
3. Run: `locust -f tests/perf/locustfile.py --headless -u 50 -r 10 -t 1m --host http://localhost:8000`.
4. Record the p90 latency in `docs/perf-sprint1.md`. The target is 2 seconds or less. If it is higher, run `EXPLAIN ANALYZE` on the search query and check the indexes.
5. Raise this as its own PR so Nishkal and Ravish can reuse the seed data.

Ravish: UI, docs and demo (15 points)

Everything under `frontend/`, plus the documentation updates and the demo video. Until the backend endpoints land, build against the contracts in SAD 4.3 and test against the Swagger page at `/docs` as endpoints appear.

Tue 27 Oct: setup

1. Install Node 20. In the repo run `npm create vite@latest frontend -- --template react`, then `npm i axios react-router-dom`.
2. Read SRS 3.1 and NF-003, SAD 4.3 and 4.5, and TC-UX-01.
3. Folder layout: `src/api`, `src/components`, `src/pages`, `src/context`.

Wed 28: app shell and auth pages (story 14)

1. `src/api/client.js`: an Axios instance with `baseURL` from `import.meta.env.VITE_API_URL`. A request interceptor adds the `Authorization: Bearer` header. A response interceptor clears the stored token and redirects to `/login` on a 401.
2. `src/context/AuthContext.jsx`: holds the user and token (store the token in `sessionStorage`), with `login()` and `logout()`.
3. `ProtectedRoute` component that redirects unauthenticated users to `/login`.
4. `Navbar` with Browse, My Listings, Profile and Logout. It must wrap or collapse cleanly at 360 px width.

5. `pages/Register.jsx` and `pages/Login.jsx`: labelled inputs, inline validation that mirrors the backend rules (email format, password of at least 8 characters with a letter and a digit), and clear messages for 409 (email exists), 401 (invalid credentials) and 429 (too many attempts, try again later).
6. `frontend/Dockerfile`: multi-stage build, Node to build and nginx to serve `dist`. Send it to Abhiram by Wednesday afternoon so `docker compose` works.

Thu 29: browse and search (story 15)

1. `pages/Browse.jsx`: search box with a 300 ms debounce, location filter, and pagination controls.
2. `components/ListingCard.jsx`: title, author, a condition badge and the location.
3. States: loading indicator, an empty state with the text No books found, and an error banner.
4. Wire it to `GET /api/v1/listings` once Manoj's PR 2 is merged (Thursday evening).

Fri 30: My Listings, Profile, documentation (stories 16, 17)

1. `pages/MyListings.jsx`: a form to create a listing (title, author, ISBN, condition select, location) with validation, and a list from `GET /listings/mine` with Edit and Delete. Delete asks for confirmation and shows the message when the server returns 409.
2. `pages/Profile.jsx`: view and edit name, department and location.
3. TC-UX-01: in Chrome DevTools use the device toolbar at 360 px and check no horizontal scroll. Count clicks for listing a book and for searching; each must be 5 or fewer.
4. RTM: in `docs/SRS.md` section 8, change the status from N to A for each Sprint 1 requirement whose tests pass (F-001 to F-004, F-010 to F-013, NF-001, NF-003, NF-004, NF-006, SR-001, SR-002, SR-004, SR-005, SR-006).
5. Create `docs/DEVIATIONS.md` as a table with columns Item, Document and section, Planned, Actual, Reason. Start it with: `token_version`, `department`, `deleted_at`, `GET /listings/mine`, in-memory rate limiter, and the chosen hosting service. Add anything the others report.

Fri evening: demo video

Aim for about 1 minute showing a working product on the deployed link:

1. Register a new user (10 s).
2. Log in (5 s).
3. Create a listing (15 s).

4. Search for it and show it in the results (10 s).
5. Edit it, then delete it (15 s).
6. End on the CI pipeline with a green run (5 s).

Open the deployed site a few minutes beforehand because free tiers can sleep. Record with OBS, Xbox Game Bar (Win+G) or QuickTime, at 1080p. Save as `/demo/sprint1.mp4`, keep it under 100 MB, and commit it through a PR on branch `BEP-<n>-demo`.

Team ground rules

Daily standup (10 minutes, same time each day)

Each person says what they finished, what they do next, and what blocks them. Update Jira before the standup so the burndown is current.

Definition of done for every story

- Code merged to `main` through a reviewed PR with the Jira key in its title.
- Tests written for each acceptance criterion and passing in CI.
- No secrets, passwords or tokens in code or logs.
- Card moved to Done in Jira on the same day.
- Any difference from the SRS or SAD added to `docs/DEVIATIONS.md`.

Merge conflicts

Pull from `main` into your branch before opening a PR. If you touch a shared file (models, `main.py`, `docker-compose.yml`), tell the team first and keep the change small.

Sprint 2 preview

Sprint 2 (2 to 7 Nov) covers requests, the matching engine, exchanges, ratings and admin. A suggested split: Manoj takes the matching engine and request endpoints, Nishkal takes exchanges, notifications and ratings, Ravish takes the matching and exchange UI plus final documentation, Abhiram takes admin, contact privacy, TLS and uptime evidence, the benchmark and the final demo. Replan on Mon 2 Nov.